

AMMC Financial Intelligence Platform

A hybrid RAG (Retrieval-Augmented Generation) system for querying annual reports of Moroccan listed companies supervised by the AMMC (Autorité Marocaine du Marché des Capitaux). Users can ask natural language questions in French or English and get structured answers backed by extracted financial data and semantic document search.

Features

- **Natural language chat** over annual reports (streaming, SSE)
 - **Hybrid retrieval**: SQL for structured financial data, ChromaDB for narrative/contextual queries
 - **Financial extraction pipeline**: extracts income statements, balance sheets, cash flows, and ratios from PDF tables
 - **Integrated PDF viewer** with CORS proxy for AMMC documents
 - **Admin analytics dashboard**: visitor counts, geographic data, demo abuse detection
 - **Demo / Admin auth**: rate-limited demo mode (5 messages), unlimited admin access
 - **Data Explorer**: browse extracted metrics by company and year
 - **Graceful degradation**: falls back to vector-only mode if PostgreSQL is unavailable
-

Tech Stack

Layer	Technology
LLM	Groq API — llama-3.3-70b-specdec
Vector DB	ChromaDB 1.5.9
Relational DB	PostgreSQL 16
Backend	FastAPI + SQLAlchemy 2.0 + Alembic

Layer	Technology
PDF processing	pdfplumber, pypdf
Frontend	React 18 + Vite 5
Containerization	Docker + Docker Compose

Project Structure

```
chatbot-ammc/
├── api/                # FastAPI app (routes, analytics)
├── db/                 # SQLAlchemy models, session, migrations
├── retrieval/          # Hybrid query router (SQL + vector)
├── extraction/         # PDF financial data extraction pipeline
├── scripts/            # Data population and validation scripts
├── frontend/          # React SPA (Vite)
├── alembic/            # Database migration versions
├── tests/              # Unit + integration tests
├── data/               # Runtime data (PDFs, chunks, metrics)
├── .chroma/            # ChromaDB persistent index
├── ammc_report_scraper/ # PDF scraper (downloads reports from ammc.ma)
├── build_index.py       # ChromaDB indexing pipeline
├── chat.py             # LLM chat + retrieval orchestration
├── extract.py          # PDF → text/chunks extraction
├── ingest_scraped.py    # Bridge: scraper output → index
├── main.py             # CLI entry point
├── config.py           # App configuration + env vars
├── docker-compose.yml  # Development stack
├── docker-compose.prod.yml # Production stack
├── Dockerfile          # Multi-stage build (Node → Python)
└── .env.example        # Environment variable template
```

Database Schema

Table	Purpose
companies	Company master (name, ticker, sector)
reports	One row per PDF (path, extraction status, hash)
financial_metrics	Individual extracted line items with confidence score
income_statements	Structured income statement (revenue, EBITDA, net income)

Table	Purpose
balance_sheets	Assets, liabilities, equity
cash_flows	Operating, investing, financing cash flows
financial_ratios	ROE, ROA, and other ratios
shareholders	Ownership structure
extraction_jobs	Extraction run metadata
extraction_errors	Per-run error log

Prerequisites

- Docker and Docker Compose
- Python 3.11+
- Node.js 20+ (for frontend development)
- A [Groq API key](#)

Getting Started

1. Configure environment

```
cp .env.example .env
```

Edit `.env` with your values:

```
# LLM
GROQ_API_KEY=your_key_here
GROQ_BASE_URL=https://api.groq.com/openai/v1
GROQ_MODEL=llama-3.3-70b-specdec

# Database
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=ammc
POSTGRES_USER=ammc
POSTGRES_PASSWORD=changeme
DATABASE_URL=postgresql://ammc:changeme@localhost:5432/ammc
```

```
# Retrieval
TOP_K=5
HISTORY_LIMIT=10

# Extraction
EXTRACTION_CONFIDENCE_THRESHOLD=0.4
EXTRACTION_DEBUG=false

# CORS (comma-separated)
CORS_ORIGINS=http://localhost:5173
```

2. Start PostgreSQL

```
docker compose up -d
```

3. Run database migrations

```
alembic upgrade head
```

4. Populate the database

```
python scripts/populate_db.py
```

5. Extract financial data from PDFs

```
# Process first 20 PDFs (for testing)
python scripts/extract_financial_data.py --limit 20

# Process all PDFs
python scripts/extract_financial_data.py
```

6. Validate extraction quality

```
python scripts/validate_extraction.py
python scripts/data_quality_report.py
```

7. Build the ChromaDB vector index

```
python build_index.py
```

8. Start the backend

```
uvicorn api.main:app --reload --host 127.0.0.1 --port 8000
```

9. Start the frontend (separate terminal)

```
cd frontend
npm install
npm run dev
# → http://localhost:5173
```

CLI Commands

main.py provides a unified CLI:

```
python main.py chat          # Interactive terminal chat
python main.py index         # Rebuild ChromaDB index
python main.py ingest        # Ingest new PDFs from data/pdfs/
python main.py serve         # Start API server on :8000
python main.py all           # Index then open chat
python main.py serve --port 8888 --host 0.0.0.0
```

Docker

Development

```
docker compose up --build
# Backend at http://localhost:8000
# Frontend at http://localhost:5173
```

Production

```
docker compose -f docker-compose.prod.yml up -d
# App at http://127.0.0.1:8010 (put behind a reverse proxy)
```

The production image is multi-stage: the frontend is compiled with Node and bundled into the Python image. No separate frontend server is needed.

API Reference

Chat

Method	Endpoint	Description
POST	/api/chat	Full response (JSON)
POST	/api/chat/stream	Streaming response (SSE)
POST	/api/search	Raw hybrid search results

Chat request body:

```
{
  "question": "Quel est le résultat net de Attijariwafa Bank en 2023 ?",
  "company": "attijariwafa-bank",
  "year": 2023,
  "history": []
}
```

Metadata

Method	Endpoint	Description
GET	/api/health	Platform status
GET	/api/reports	Indexed report list
GET	/api/companies	Company list
GET	/api/companies/db	Companies from DB (with ?sector= filter)

Financial Data

Method	Endpoint	Description
GET	/api/companies/{slug}/metrics	Extracted metrics for a company/year
GET	/api/companies/{slug}/financials	Full financial statements
GET	/api/companies/{slug}/shareholders	Shareholder structure

PDF Proxy

Method	Endpoint	Description
GET	/api/pdf?url=<ammc_url>	Proxies a PDF from ammc.ma (bypasses browser CORS)

Admin & Analytics

Method	Endpoint	Description
POST	/api/track	Track a frontend event
GET	/api/analytics	Summary (visits, events, demo usage)
GET	/api/analytics/visitors	Visitor details with IP and location
POST	/api/ingest	Trigger re-indexing

How Retrieval Works

Queries are routed **without an LLM call** using keyword matching:

Mode	Trigger	Data source
SQL	Financial keywords (résultat net, total actif, EBITDA...)	PostgreSQL
VECTOR	Narrative keywords (risque, stratégie, gouvernance...)	ChromaDB
HYBRID	Causal keywords (pourquoi, impact, facteur...)	Both
VECTOR (default)	Ambiguous query	ChromaDB

The LLM receives the merged context and is instructed to answer only from retrieved data. If data is absent it replies "données non disponibles" rather than hallucinating.

Data Ingestion Pipeline

```
ammc.ma
  ↓ ammc_report_scraper (downloads PDFs)
reports_manifest.csv
  ↓ ingest_scraped.py (copies to data/pdfs/)
extract.py (pdfplumber → 300-word chunks, 50-word overlap)
  ↓ data/chunks/{stem}.json
build_index.py
  ↓ ChromaDB (.chroma/)

extraction/financial_extractor.py (table → metrics, French number normalizer)
  ↓ scripts/extract_financial_data.py
PostgreSQL (income_statements, balance_sheets, financial_metrics...)
```

Tests

```
python -m pytest tests/ -v
```

- `test_normalization.py` — French number parsing ("1 245,50 MDH" → 1245.5)
- `test_validation.py` — Balance sheet / income statement validation rules
- `test_router.py` — Query routing logic
- `test_sql_retriever.py` — SQL queries (skipped if PostgreSQL is unavailable)

Authentication

Two hardcoded accounts exist in `frontend/src/components/Login.jsx`:

Role	Behaviour
Demo	5 messages per device (enforced client-side + server rate-limit)
Admin	Unlimited access + analytics dashboard

Credentials are stored in `sessionStorage`. This is suitable for internal/demo use; replace with a proper auth system for public deployment.

Environment Variables Reference

Variable	Default	Description
<code>GROQ_API_KEY</code>	—	Required. Groq API key
<code>GROQ_BASE_URL</code>	Groq endpoint	OpenAI-compatible base URL
<code>GROQ_MODEL</code>	<code>llama-3.3-70b-specdec</code>	Model name
<code>DATABASE_URL</code>	—	SQLAlchemy connection string
<code>POSTGRES_*</code>	—	Host, port, db, user, password

Variable	Default	Description
TOP_K	5	ChromaDB results per query
HISTORY_LIMIT	10	Chat turns kept in context
EXTRACTION_CONFIDENCE_THRESHOLD	0.4	Min confidence to persist a metric
EXTRACTION_DEBUG	false	Verbose extraction logs
CORS_ORIGINS	http://localhost:5173	Allowed frontend origins
API_TOKEN	—	Optional bearer token for API access
RATE_LIMIT_PER_MIN	20	Chat requests per IP per minute
WATCH_ENABLED	true	Auto-ingest on new PDFs (disable in prod)

License

Private project — AMMC / internal use.